

INVENTAIRE DE RÉFÉRENCE

Vector

Commando IA — Analyse de Données

Document	Référence à consulter quotidiennement
Plateforme	A.I. COMMANDOS
Stack	FastAPI · Next.js · PostgreSQL+pgvector · OpenAI
Pipelines	Data Analysis (CSV/XLSX) + RAG (PDF/DOCX) — Option B
Durée	6 semaines · 24 jours travaillés
Mode	Construction from scratch
Date	Juin 2026

Tout ce qui est verrouillé avant la première ligne de code : outils, capacités, design, volumes, pipelines, tables, livrables, périmètre.

Table des matières

1. Stack technique complète	3
2. Outils installés sur la machine	6
3. Spécifications produit verrouillées	7
4. Design system verrouillé	9
5. Volumes et contraintes	11
6. Les deux pipelines (option B)	12
7. Tables de base de données prévues	15
8. Livrables prévus	16
9. Inputs encore en attente	17
10. Hors périmètre — ce qu'on ne fait pas	18
11. Plan ajusté pour intégrer le RAG	19
12. Méthode de travail	20
13. Récapitulatif final	21

1. Stack technique complète

Tout ce qui sera utilisé pendant le projet. Chaque outil a son rôle précis dans l'architecture, qu'on développera au fil des semaines.

1.1 Backend (Python)

Outil	Rôle
Python 3.11	Langage du backend
FastAPI	Framework web async, gère toutes les routes HTTP et WebSockets
Uvicorn	Serveur ASGI qui exécute FastAPI
Pydantic v2	Validation et typage des données entrantes et sortantes
SQLAlchemy 2.0 (async)	ORM pour parler à Postgres en objets Python
asyncpg	Driver asynchrone PostgreSQL pour SQLAlchemy
Alembic	Gestion des migrations de schéma de base de données
python-jose	Génération et vérification des JWT pour l'authentification
bcrypt / passlib	Hash sécurisé des mots de passe utilisateurs
WebSockets (FastAPI)	Streaming temps réel des réponses du LLM
Celery	Tâches asynchrones en arrière-plan (embeddings, parsing lourd)
Redis 7	Broker de messages pour Celery + cache rapide
pandas	Lecture et calcul sur les données tabulaires (CSV/Excel)
openpyxl	Parser des fichiers Excel (.xlsx)
pypdf	Extraction du texte des PDF pour le pipeline RAG
python-docx	Extraction du texte des documents Word pour le pipeline RAG
Jinja2	Templating HTML pour la génération des dashboards
tiktoken (cl100k_base)	Comptage exact des tokens, indispensable pour le chunking
openai (1.54+)	SDK officiel pour communiquer avec l'API OpenAI
pgvector (client Python)	Intégration de pgvector dans SQLAlchemy

1.2 Frontend (TypeScript)

Outil	Rôle
Node.js 24	Runtime JavaScript côté serveur (dev tools)
pnpm 11	Gestionnaire de paquets performant, économe en disque
Next.js 16	Framework React fullstack (App Router, Server Components)
React 19	Bibliothèque d'interface utilisateur
TypeScript	JavaScript typé pour éviter les erreurs runtime
Tailwind CSS v4	Framework CSS utility-first
Radix UI	Composants accessibles non-stylés (Modal, Dropdown, Tabs...)
TanStack Query	Gestion des appels API, cache, mutations
TanStack Table	Tableaux puissants pour KPI et données
Zustand	Store d'état global léger (alternative à Redux)
react-markdown + remark-gfm	Rendu du Markdown streamé par Vector
ApexCharts	Graphiques interactifs pour les dashboards
html2pdf.js	Conversion HTML → PDF directement dans le navigateur
lucide-react	Bibliothèque d'icônes légère
date-fns	Manipulation et formatage des dates
react-dropzone	Composant d'upload de fichiers en drag-and-drop

1.3 Base de données et infrastructure

Outil	Rôle
PostgreSQL 16	Base de données relationnelle principale
pgvector (extension)	Stockage et recherche de vecteurs (embeddings)
Docker Desktop 29	Conteneurisation de l'environnement de développement
Docker Compose	Orchestration des services en local (db, redis, backend, worker)

1.4 IA et modèles LLM (OpenAI)

Modèle	Usage	Coût indicatif
GPT-4o	Conversations, génération d'analyses, JSON structuré, recommandations	5\$/M tokens in, 15\$/M out
GPT-4o-mini	Tâches simples : validation, classification, retry sur JSON cassé	0,15\$/M in, 0,60\$/M out
text-embedding-3-small	Vectorisation des chunks documents (dimension 1536)	0,02\$/M tokens

✓ Stratégie de coût

Tous les appels secondaires (validation, classification, suggestions courtes) utilisent GPT-4o-mini, environ 30 fois moins cher que GPT-4o. Seules les générations principales (analyses, dashboards JSON, recommandations) utilisent GPT-4o.

2. Outils installés sur la machine

État vérifié de la machine de développement. Tout est en place pour démarrer.

Outil	Version	Statut	Notes
Docker Desktop	29.5.3	✓ Installé	Très récent, parfait
Python	3.11.9	✓ Installé	À invoquer via <code>py -3.11</code> sur Windows
Python 3.14	3.14.4	Présent mais non utilisé	Trop récent, bibliothèques pas toutes compatibles
Node.js	24.15.0	✓ Installé	Au-dessus du minimum requis
pnpm	11.8.0	✓ Installé	Gestionnaire de paquets choisi
Git	2.54.0 (Windows)	✓ Installé	OK
VS Code	—	Supposé installé	Éditeur recommandé

2.1 Extensions VS Code recommandées

- **Python** (Microsoft) — support Python complet
- **Pylance** (Microsoft) — analyse statique et complétion intelligente
- **Tailwind CSS IntelliSense** — autocomplétion des classes Tailwind
- **ESLint** — linting JavaScript/TypeScript
- **Prettier** — formatage automatique du code
- **Docker** (Microsoft) — gestion des conteneurs depuis l'éditeur
- **GitLens** — vue avancée de l'historique Git

3. Spécifications produit verrouillées

3.1 Capacités officielles de Vector

Tirées du catalogue PDF officiel de la plateforme A.I. COMMANDOS, ces 4 capacités définissent l'identité de Vector :

- **Construire des dashboards KPI automatisés** — l'usage principal
- **Analyser les performances marketing et commerciales** — analyses spécialisées
- **Identifier les clients et segments les plus rentables** — segmentation RFM
- **Générer des recommandations data-driven automatiques** — toujours actif

3.2 Les 5 actions rapides Vector

Les 5 boutons cliquables qui apparaissent sur la page de Vector, comme pour les autres Commandos (Scout en a 5, Sentinel en a 5, Lex en a 6). Validés avec l'encadrant.

#	Action	Ce qu'elle produit
1	Génère un dashboard KPI général	Dashboard avec KPI suggérés, analyse en sections, recommandations finales
2	Analyse les performances marketing	Dashboard focalisé canaux d'acquisition, ROI, CAC, taux de conversion + PDF téléchargeable
3	Analyse les performances commerciales	Dashboard focalisé CA, panier moyen, top produits, performance par commercial + PDF
4	Identifie les clients et segments rentables	Segmentation RFM (Récence/Fréquence/Montant), top clients par valeur + PDF
5	Détecte tendances et anomalies	Variations notables période sur période, anomalies statistiques (z-score, IQR)

3.3 Section Recommandations — automatique partout

Quelle que soit l'action déclenchée (les 5 ci-dessus), **chaque** dashboard généré se termine systématiquement par une section « Recommandations data-driven ». Cette section n'est pas un bouton, c'est un comportement par défaut.

Caractéristiques de la section :

- 3 à 5 recommandations par dashboard
- Chaque recommandation est **priorisée** : Haute, Moyenne ou Basse
- **Impact attendu** chiffré quand c'est possible (ex. « +12% CA »)

- Formulation **actionable** : pas « il faudrait mieux performer », mais « concentrer les efforts sur la région Sud-Est »
- Générée par GPT-4o à partir de l'analyse complète du dashboard

✓ Décision verrouillée

La section Recommandations sera présente sur 100% des dashboards produits par Vector, que ce soit via les 5 actions rapides ou via le chat libre. C'est un signature de l'agent.

4. Design system verrouillé

Basé sur l'analyse de la plateforme live (hq.ai-commandos.com) et des mockups officiels fournis par l'encadrant.

4.1 Couleur signature Vector

Chaque agent de la plateforme a sa couleur signature, visible notamment sur la bordure gauche des cartes d'actions rapides. Voici les couleurs observées :

Agent	Couleur signature
Sentinel (Veille)	Violet
Scout (Recrutement)	Cyan / Teal pâle
Lex (Juridique)	Jaune / Or
Hunter (Prospection)	Rouge
Vector (Analyse de données)	Vert profond ← le nôtre

✓ Couleur Vector

Vert profond style sapin/forêt, observé sur la page d'onboarding du ZIP UI. Le hex exact sera extrait au J1 directement depuis l'image. Estimation actuelle : **#2D8659**.

4.2 Palette globale de la plateforme

Élément	Couleur	Hex approximatif
Sidebar	Bleu marine quasi noir	#0F1419
Fond principal (dark)	Noir profond	#0A0E14
Accent principal de la plateforme	Teal / cyan vif	#14B8A6
Accent Vector (signature)	Vert profond	#2D8659
Texte principal	Blanc cassé	#F5F5F0
Texte secondaire	Gris clair	#9CA3AF
Bordures subtiles	Gris foncé	#1F2937
Bouton Web actif / envoi	Teal sur fond noir	#14B8A6

4.3 Mode par défaut

Confirmation depuis la plateforme live : le **mode sombre est par défaut**. On code donc en dark theme par défaut. Un toggle clair/sombre sera ajouté en bonus si le temps le permet, mais ce n'est pas prioritaire.

4.4 Layout d'une page agent

Structure observée sur les pages Sentinel, Scout et Lex. C'est ce qu'on reproduit pour Vector.

Composants de gauche à droite, de haut en bas :

- **Sidebar gauche étroite** (icônes seulement) : Accueil, Commandos IA, Discussions, Documents, QG
- **Header haut** : logo « A.I. COMMANDOS » + boutons « Suggestions » et « Nouvelles questions » avec badges + icônes droite (invitation, cadeau, paramètres, profil)
- **Panel agent central gauche** : avatar Vector + nom + rôle + bouton « Nouveau Chat » + liste des discussions passées
- **Main content** : titre « Commando IA Vector », sous-titre italique « à votre service ! », accroche « Quelle est la mission aujourd'hui ? »
- **Section ACTIONS RAPIDES** : grille 3+2 de cartes cliquables, bordure gauche verte
- **Section CHAT LIVE en bas** : input « Poser une question à Vector... » + toggle Web + micro + bouton d'envoi rond teal

4.5 Anatomie d'une carte d'action rapide

- Icône ■ (éclair) en haut à gauche
- **Titre en gras** (ex. « Génère un dashboard KPI général »)
- **Description courte** en gris (1 à 2 lignes)
- **Bordure gauche verte** (signature Vector)
- Lien « **Exécuter** → » en bas à droite, en vert
- Fond très sombre, légèrement plus clair que le fond de page
- Hover : légère élévation et bordure plus visible

5. Volumes et contraintes confirmés

Toutes les valeurs ci-dessous ont été validées par l'encadrant ou résultent d'une décision d'architecture argumentée.

Paramètre	Valeur	Implication
Volume typique d'un dataset	100 000 lignes	pandas charge sans souci en RAM (<1 sec)
Taille fichier max (dataset)	100 Mo	Marge x3 par rapport au cas standard
Volume typique d'un document RAG	Quelques pages à 200 pages	Génère 5 à 400 chunks selon document
Taille fichier max (document RAG)	10 Mo	Calqué sur la vraie plateforme
Formats datasets	.csv, .xlsx, .xls	pandas gère via read_csv et read_excel
Formats documents RAG	.pdf, .docx, .txt, .md	pypdf, python-docx, lecture directe
Anonymisation OpenAI	Non requise	Simplifie le code, mais on envoie quand même un résumé pour le coût
Charte graphique PDF	Aucune (liberté)	On choisira un look professionnel sobre
Environnement de staging	Aucun	Demos depuis le poste local
Format des démos vidéo	Loom	Extension Chrome ou app desktop, lien partageable
Durée des démos hebdo	3 à 5 minutes	8 à 10 minutes pour la démo finale S6

5.1 Implications du volume 100k lignes

- **Pas besoin de techniques exotiques** (sampling, streaming, DuckDB) : pandas tient en mémoire
- Le **profilage statistique** prend 2 à 3 secondes
- Les **calculs KPI** restent instantanés
- Pour les **graphiques temporels**, on agrège par jour/semaine/mois côté backend (jamais 100k points envoyés au frontend)
- Pour les **colonnes catégorielles** à forte cardinalité (>50 valeurs uniques), on garde le top-20 + total « Autres » dans le résumé envoyé au LLM
- **Temps de génération d'un dashboard** : 8 à 15 secondes (vs 5-10 sur petit dataset)

- Message de chargement adapté : « Analyse en cours sur 100 000 lignes... »

6. Les deux pipelines (option B retenue)

Décision prise d'implémenter **les deux pipelines**. Vector combinera donc l'analyse de données structurées **et** l'exploitation de contexte textuel via RAG. Cela enrichit considérablement la valeur de l'agent.

6.1 Pipeline A — Data Analysis

C'est le cœur de Vector. Entrée : un fichier CSV ou Excel. Sortie : un dashboard HTML interactif avec PDF téléchargeable.

Les étapes du pipeline A :

Étape	Outil	Sortie
1. Upload du fichier	FastAPI multipart + react-dropzone	Fichier stocké
2. Parsing	pandas (read_csv, read_excel)	DataFrame en mémoire
3. Profilage statistique	pandas describe + value_counts + corr	Résumé compact (~1 page JSON)
4. Génération de l'analyse	GPT-4o avec response_format=json_schema	JSON structuré
5. Calcul des KPI	pandas sur les vraies données	Valeurs exactes
6. Rendu HTML	Jinja2 + template dashboard.html.j2	HTML complet
7. Affichage	Next.js iframe srcDoc	Dashboard visible
8. Export PDF (à la demande)	html2pdf.js dans le navigateur	PDF téléchargé

6.2 Pipeline B — RAG Documents

Pipeline parallèle pour le contexte textuel. Entrée : PDF, Word, texte. Sortie : chunks vectorisés indexés, prêts à être injectés dans les conversations Vector.

Phase d'ingestion (au moment de l'upload) :

Étape	Outil	Sortie
1. Upload du document	FastAPI multipart + react-dropzone	Fichier stocké
2. Extraction du texte	pypdf (PDF), python-docx (Word)	Texte brut
3. Chunking	Chunker maison, 500 tokens, 50 chevauchement	Liste de chunks
4. Comptage tokens	tiktoken (cl100k_base)	Nombre de tokens par chunk
5. Embedding (async Celery)	text-embedding-3-small via OpenAI	Vecteur(1536) par chunk
6. Stockage	PostgreSQL + pgvector, index IVFFlat cosine	Chunks indexés

Phase d'utilisation (à chaque conversation Vector) :

Étape	Outil	Sortie
1. Question utilisateur	Chat live ou action rapide	Texte de la question
2. Embedding de la question	text-embedding-3-small	Vecteur(1536)
3. Recherche top-5	pgvector cosine distance	5 chunks les plus proches
4. Filtrage seuil ≤ 0.7	Filtre Python	0 à 5 chunks conservés
5. Formatage en contexte	Injector maison	Texte prêt à injecter
6. Injection dans le prompt	Prompt enrichi pour GPT-4o	Réponse contextualisée

6.3 Quand Vector utilise quoi

Cas d'usage	Pipeline utilisé
Utilisateur uploade un CSV et clique sur une action rapide	A (data) — RAG ignoré
Utilisateur pose une question textuelle (chat) sans dataset	B (RAG) seul si documents disponibles
Utilisateur a un dataset + des documents et pose une question	A + B combinés (le top !)
Utilisateur clique « Analyse marketing » avec dataset + docs disponibles	A principal, B en contexte enrichi

✓ La combinaison A+B = le saint Graal

C'est dans le cas où Vector a accès à **la fois** aux données chiffrées (dataset) et au contexte business (documents) que ses recommandations deviennent vraiment puissantes. Exemple : Vector voit que le CA baisse de 12% sur la région Sud (donnée), et il lit dans le plan stratégique 2026 que la région Sud est prioritaire (contexte). Il peut alors recommander de manière éclairée : « Action urgente, écart avec la stratégie définie ».

7. Tables de base de données prévues

Schéma prévu. Le détail des colonnes (types, contraintes, indexes) sera défini au Jour 2 de la Semaine 1.

Table	Pipeline	Rôle
users	Tous	Comptes utilisateurs avec auth JWT
agents	Tous	Définition de Vector (et futurs agents)
conversations	Tous	Threads de chat utilisateur ↔ agent
messages	Tous	Messages échangés dans les conversations
datasets	A (data)	Métadonnées des CSV/XLSX uploadés
dataset_columns	A (data)	Colonnes détectées avec types et stats
kpis	A (data)	Définitions de KPI configurés par l'utilisateur
kpi_values	A (data)	Valeurs calculées (snapshots) des KPI
dashboards	A (data)	Dashboards générés (HTML + JSON source)
analyses	A (data)	Résultats des analyses spécialisées
documents	B (RAG)	Métadonnées des PDF/DOCX uploadés
chunks	B (RAG)	Chunks vectorisés (colonne embedding Vector(1536))

7.1 Relations principales

- **users** → **conversations** (1:N) — un utilisateur a plusieurs conversations
- **conversations** → **messages** (1:N) — une conversation contient plusieurs messages
- **users** → **datasets** (1:N) — un utilisateur uploade plusieurs datasets
- **datasets** → **kpis** (1:N) — un dataset a plusieurs KPI configurés
- **datasets** → **dashboards** (1:N) — un dataset peut produire plusieurs dashboards
- **users** → **documents** (1:N) — un utilisateur uploade plusieurs documents
- **documents** → **chunks** (1:N) — un document est découpé en plusieurs chunks

7.2 Index spéciaux

- **chunks.embedding** — index IVFFlat avec opérateur cosinus (vector_cosine_ops)
- **messages.conversation_id** — index B-tree classique
- **datasets.user_id, documents.user_id** — pour filtrer par utilisateur

8. Livrables prévus

8.1 Démonstrations vidéo Loom hebdomadaires

Semaine	Démo	Contenu
S1	Démo 1	Stack opérationnelle, Vector apparaît, répond en streaming
S2	Démo 2	Upload CSV, profil statistique, première analyse textuelle
S3	Démo 3	KPI suggérés automatiquement, édition manuelle, recalcul
S4	Démo 4	Dashboard HTML « KPI général » avec Recommandations
S5	Démo 5	5 actions rapides + PDF téléchargeable + RAG opérationnel
S6	Démo finale	Démonstration complète scénarisée (8-10 min)

8.2 Rapport professionnel final

Document de 30 à 50 pages, livré en fin de semaine 6. Plan recommandé :

- Page de garde, remerciements, table des matières
- Introduction : présentation de l'entreprise et de la plateforme A.I. COMMANDOS
- Objectifs du stage et périmètre de Vector
- État de l'art : LLM en entreprise, RAG, dashboards data-driven
- Architecture technique choisie avec diagrammes
- Justification détaillée des choix techniques (Jinja2 plutôt que HTML LLM, pandas pour calculs, etc.)
- Réalisation : déroulé semaine par semaine, captures et extraits de code clés
- Résultats : métriques (temps de génération, coût LLM par dashboard, nombre de tokens)
- Limites identifiées et perspectives (intégration Core, multi-tenant, etc.)
- Conclusion : bilan personnel et technique
- Annexes : extraits de code significatifs, glossaire, références

9. Inputs encore en attente

Tout ce qui doit encore être fourni par l'encadrant. Bien suivre les dates limites pour éviter de bloquer le projet.

Item	Date limite	Bloquant si manquant
Format HTML imposé du dashboard	Fin S3	Oui pour S4
Dataset CSV d'exemple	Fin S1	Oui pour S2
Dataset XLSX d'exemple	Fin S2	Non, le CSV suffit pour démarrer
Validation des prompts système de Vector	S1 J3	Non, on itère
Confirmation du choix de couleur Vector exact	S1 J1	Non, on extrait du ZIP

■ Si en retard

Si à la fin de la semaine 3 le format HTML n'est pas reçu, on continue avec un format « par défaut » professionnel et on l'adapte ensuite quand il arrive. On ne se bloque jamais.

10. Hors périmètre — ce qu'on ne fait pas

Cette liste est aussi importante que celle de ce qu'on fait. Elle protège du **scope creep** (élargissement non contrôlé du périmètre) et permet de tenir les 6 semaines.

Item	Pourquoi on l'exclut
Code source de la plateforme A.I. COMMANDOS	Décision validée : on construit Vector en autonomie depuis zéro
Intégration au module Core de la plateforme	L'encadrant le précisera après le stage si l'entreprise le souhaite
Stripe et facturation	Pas pertinent pour démontrer Vector
Envoi d'emails transactionnels (SMTP)	Pas utile pour le MVP, ajoutable plus tard
Sentry monitoring	Bonus de la S6 si du temps reste
Multi-tenant et workspaces	Mono-utilisateur suffit pour démontrer le produit
Intent router avec gpt-4o-mini	Vector est l'unique agent, pas de routage nécessaire
Les 12 autres agents (Scout, Lex, Nova, etc.)	Vector est l'objet du stage
Authentification OAuth, SSO, Google Sign-In	JWT email/password suffit pour la démo
Tests E2E avec Playwright	Bonus S6, tests unitaires Pytest suffisent en premier
Internationalisation (i18n)	Interface en français uniquement
Mode clair / toggle dark-light	Dark seulement, bonus possible S6

11. Plan ajusté pour intégrer le RAG (option B)

Ajout du pipeline RAG dans le plan initial. Le RAG est principalement construit en semaine 5, mais sa fondation est posée dès la semaine 2.

Semaine	Ajout RAG par rapport au plan initial
S1	Tables documents et chunks créées dès la migration initiale
S2	Page Documents (UI) + endpoint upload basique (pas encore d'ingestion)
S3	Rien de nouveau côté RAG (focus KPI)
S4	Rien de nouveau côté RAG (focus dashboards)
S5	Pipeline RAG complet : parser, chunker, embedder, stockage, retriever, injecteur. C'est la grosse semaine RAG. La semaine est partagée 50/50 entre RAG et analyses spécialisées.
S6	Intégration finale : Vector utilise systématiquement les documents pertinents quand il analyse les données. Tests de bout en bout du combo A+B.

11.1 Détail de la semaine 5 revisitée

- **J1** — Pipeline RAG côté backend : parser PDF/DOCX, chunker, embedder, stockage pgvector. Tâche Celery pour l'embedding async.
- **J2** — Retriever pgvector (top-5 cosine) et injecteur dans les prompts Vector. Action « Analyse marketing » avec PDF + intégration RAG.
- **J3** — Action « Analyse commerciale ». Page Documents complète côté frontend (upload, liste, suppression, statut d'ingestion).
- **J4** — Action « Identifier clients » (RFM) + Action « Détecter tendances » + Démo 5.

✓ La densité de la semaine 5

C'est la semaine la plus chargée du plan. Si on prend du retard sur l'une des actions spécialisées, on en reporte une seule sur le S6 J1, pas plus. Le RAG, lui, n'est pas négociable car il conditionne la démo finale.

12. Méthode de travail

Cette méthode garantit que tu comprends chaque ligne de code écrite. C'est essentiel pour le rapport final et pour ta propre compétence acquise pendant le stage.

12.1 Les 6 principes

Principe	Pratique
Le « pourquoi » avant le « comment »	Avant chaque étape, on explique le problème et la solution choisie
Code expliqué bloc par bloc	Aucun pavé de code lâché sans explication. Chaque import, décorateur, pattern est commenté
Test à chaque étape	On lance, on vérifie que ça marche, puis on passe à la suite
Stop si tu ne comprends pas	Tu dis « explique-moi cette ligne », on n'avance pas avant que tu sois à l'aise
Bilan rapide en fin de journée	Ce qui marche, ce qui coince, ce qu'on attaque demain. 5 minutes maximum.
Démo Loom en fin de semaine	Tu enregistres une démo de 3 à 5 minutes et tu l'envoies à l'encadrant

12.2 Bonnes habitudes à prendre

- **Commit Git quotidien** avec message descriptif (« feat: add user model » plutôt que « update »)
- **Petit journal de bord** dans un fichier JOURNAL.md : ce que tu as fait, ce qui a coincé. Servira pour le rapport.
- **Ne jamais commit ton fichier .env** (contient la clé OpenAI)
- **Tester avec plusieurs datasets** pour vérifier la robustesse
- **Pas de copier-coller aveugle** : tu écris (ou retapes) le code, ça t'aide à le mémoriser
- **Demander de l'aide tôt** : si tu bloques 30 minutes sur un truc, tu me demandes

13. Récapitulatif final

En une phrase, le projet :

✓ Le projet en une phrase

Construire **Vector**, un agent IA d'analyse de données autonome, qui combine **analyse de données structurées** (CSV/Excel → dashboards HTML avec KPI configurables, graphiques, PDF téléchargeable) **et RAG textuel** (PDF/DOCX → contexte business injecté dans les prompts), produit toujours une section « Recommandations data-driven », avec une UI dark theme + accent vert calquée sur la plateforme A.I. COMMANDOS, livré en 6 semaines avec démos Loom hebdomadaires et rapport pro final.

Les chiffres clés du projet

Métrique	Valeur
Durée totale	6 semaines
Jours travaillés	24 jours (4 par semaine)
Volume cible des datasets	100 000 lignes
Demos vidéo Loom	6 (une par semaine)
Pipelines à construire	2 (Data Analysis + RAG)
Actions rapides Vector	5 boutons
Capacités officielles	4 (du catalogue)
Tables de base de données	12
Modèles OpenAI utilisés	3 (GPT-4o, GPT-4o-mini, text-embedding-3-small)
Pages de rapport final visées	30 à 50

Prochaine étape immédiate

Confirmer que cet inventaire est complet et correct, puis attaquer le **Jour 1 Semaine 1** : créer la structure complète du projet (dossiers backend, frontend, docker-compose), initialiser les dépendances, vérifier que tout démarre proprement.

✓ Tu es prêt

Tout est défini. Tous les outils sont en place. Toutes les décisions sont prises. Il ne reste qu'à construire. À chaque étape, on prendra le temps de bien faire et bien comprendre. Bon stage !